hdoc day 34 by trishil chauhan

Aim

To write a menu-driven C program using separate functions and variable-length 2D integer arrays to verify the left and right distributive properties of matrix multiplication over addition.

Properties and dimensions

Left: A(B+C) = AB+AC, where A is m x n and B, C are n x p. Both sides are m x p.

Right: (A+B)C = AC+BC, where A, B are m x n and C is n x p. Both sides are m x p.

C Program

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c], char name) {
    printf("Enter elements of %c (%d x %d):\n", name, r, c);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void addMatrix(int r, int c, int a[r][c], int b[r][c],
               int result[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            result[i][j] = a[i][j] + b[i][j];
}

void multiplyMatrix(int r, int n, int c, int a[r][n],
                    int b[n][c], int result[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) {
            result[i][j] = 0;
            for (int k = 0; k < n; k++)
                result[i][j] += a[i][k] * b[k][j];
        }
}

int equalMatrix(int r, int c, int a[r][c], int b[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            if (a[i][j] != b[i][j]) return 0;
    return 1;
}

void printMatrix(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) printf("%6d", a[i][j]);
        printf("\n");
    }
}

void leftDistributive(void) {
    int m, n, p;

    printf("Enter m, n, p (positive): ");
    if (scanf("%d%d%d", &m, &n, &p) != 3 ||
        m <= 0 || n <= 0 || p <= 0 || m > 20 || n > 20 || p > 20) {
        printf("Invalid dimensions (allowed: 1 to 20).\n");
        return;
```

```
    }
    int A[m][n], B[n][p], C[n][p];
    int BC[n][p], AB[m][p], AC[m][p], lhs[m][p], rhs[m][p];
    readMatrix(m, n, A, 'A');
    readMatrix(n, p, B, 'B');
    readMatrix(n, p, C, 'C');
    addMatrix(n, p, B, C, BC);
    multiplyMatrix(m, n, p, A, BC, lhs);
    multiplyMatrix(m, n, p, A, B, AB);
    multiplyMatrix(m, n, p, A, C, AC);
    addMatrix(m, p, AB, AC, rhs);
    printf("\nA*(B+C):\n"); printMatrix(m, p, lhs);
    printf("(A*B)+(A*C):\n"); printMatrix(m, p, rhs);
    printf("Left distributive property: %s\n",
           equalMatrix(m, p, lhs, rhs) ? "VERIFIED" : "NOT VERIFIED");
}

void rightDistributive(void) {
    int m, n, p;
    printf("Enter m, n, p (positive): ");
    if (scanf("%d%d%d", &m, &n, &p) != 3 ||
        m <= 0 || n <= 0 || p <= 0 || m > 20 || n > 20 || p > 20) {
        printf("Invalid dimensions (allowed: 1 to 20).\n");
        return;
    }
    int A[m][n], B[m][n], C[n][p];
    int AB[m][n], AC[m][p], BC[m][p], lhs[m][p], rhs[m][p];
    readMatrix(m, n, A, 'A');
    readMatrix(m, n, B, 'B');
    readMatrix(n, p, C, 'C');
    addMatrix(m, n, A, B, AB);
    multiplyMatrix(m, n, p, AB, C, lhs);
    multiplyMatrix(m, n, p, A, C, AC);
    multiplyMatrix(m, n, p, B, C, BC);
    addMatrix(m, p, AC, BC, rhs);
    printf("\n(A+B)*C:\n"); printMatrix(m, p, lhs);
    printf("(A*C)+(B*C):\n"); printMatrix(m, p, rhs);
    printf("Right distributive property: %s\n",
           equalMatrix(m, p, lhs, rhs) ? "VERIFIED" : "NOT VERIFIED");
}

int main(void) {
    int choice;
    do {
        printf("\n--- MATRIX DISTRIBUTIVE PROPERTIES ---\n");
        printf("1. Verify left distributive property\n");
        printf("2. Verify right distributive property\n");
        printf("3. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid input.\n");
            return 1;
        }
        switch (choice) {
            case 1: leftDistributive(); break;
            case 2: rightDistributive(); break;
            case 3: printf("Exiting program.\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }
    } while (choice != 3);
    return 0;
}
```

Sample verification

Choose option 1 and enter m = n = p = 2, A = [[1, 2], [3, 4]], B = [[2, 0], [1, 2]], C = [[1, 1], [0, 1]]. Both A(B+C) and AB+AC produce [[6, 6], [14, 14]], so the left distributive property is verified.

Choose option 2 with the same matrices. Both (A+B)C and AC+BC produce [[3, 7], [4, 12]], so the right distributive property is verified.

Result

The program computes both sides independently, displays the resulting matrices, and compares corresponding entries to verify each distributive property.